Preliminary Design Review & Prototype

Multi-Author Writing Style Change Detection

Tshephang Phuti-A-Nkutu Matlala

Academy of Computer Science and Software Engineering
University of Johannesburg
South Africa

Outline

What is Author Style Change Detection?

- Detecting points in a document where the **writing style changes**.
- Implies the presence of **multiple authors**.
- Applications:
  - Plagiarism and contract cheating detection.
  - Forensic linguistics and authorship attribution.
  - Analysis of collaborative writing (e.g., Wikipedia).
- This project develops a **web-based tool** that automates detection using stylometric features and machine learning.

Problem Statement

Challenges

- Subtle stylistic differences between authors.
- Lack of large, annotated multi-author datasets.
- Need for **interpretable** results, not just black-box predictions.

Problem Statement

Challenges

- Subtle stylistic differences between authors.
- Lack of large, annotated multi-author datasets.
- Need for **interpretable** results, not just black-box predictions.

Goal

Build a **proof-of-concept prototype** that:

- Extracts 31 stylometric features from sentence pairs.
- Trains a classifier to predict author switches.
- Provides confidence scores and explanatory features.
- Validates the feasibility of the processing pipeline and user interaction.

Four-Stage Pipeline

Data → Features → Model → Evaluation

Data
Text upload, sentence segmentation, input validation.

Features
Stylometric feature extraction from each sentence pair.

Model
Classifier training (LR, SVM, k-NN, MLP) and author-switch prediction.

Evaluation
Cross-validation, metrics, model comparison.

- **Separation of concerns**: each package can be modified independently.
- Aligns with the conceptual class diagram (next slide).

Architecture Justification

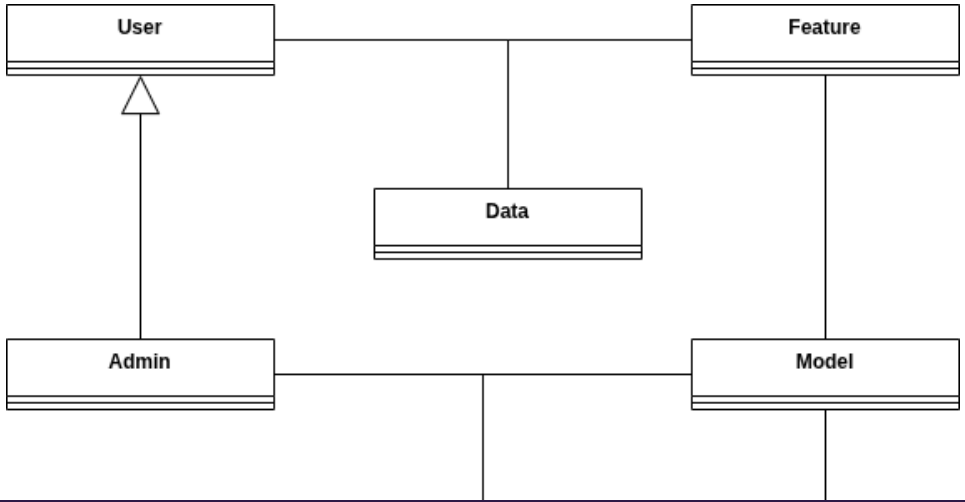
The four-package decomposition provides:

- **Data** – isolates sentence segmentation and input validation changes.
- **Features** – allows new stylometric features without affecting other components.
- **Model** – enables independent replacement or extension of the classification algorithm.
- **Evaluation** – permits separate evolution of metrics and reporting.

Benefits:

- Improved maintainability.
- Simplified unit and integration testing.
- Direct alignment with the domain model (conceptual class diagram).

Conceptual Class Diagram



Use Case Traceability

Use Case	Related FRs	Validating Tests	Status
UC1: Upload text for analysis	FR1–FR3	UT-06–11, IT-01–03	**Implemented**
UC2: View predictions	FR5–FR6	IT-04–06, ST-01–02	**Implemented**
UC3: Interpret confidence	FR7	ST-01, UAT	**Partial**
UC4: Train model	FR8	IT-07–08, ST-04	**Implemented**
UC5: Compare model performance	FR9	IT-11	**Partial**
UC6: Login and authentication	Auth	UT-01–05	**Partial**

Prototype Overview

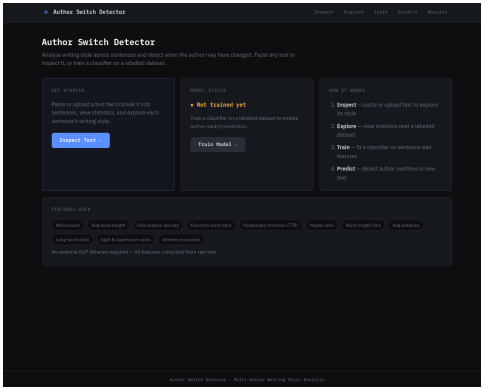
- Implemented as a **Flask-based web application** (Python).
- Technologies: Flask, NumPy, scikit-learn, HTML/CSS.
- Pages:
  - **Home** – model status, quick-start links.
  - **Inspect** – upload text, view per-sentence features.
  - **Explore** – dataset statistics and examples.
  - **Train** – classifier selection, training with live log.
  - **Predict** – submit text, probability threshold, results.
- Runs locally via `python app.py`; `http://localhost:5000`.

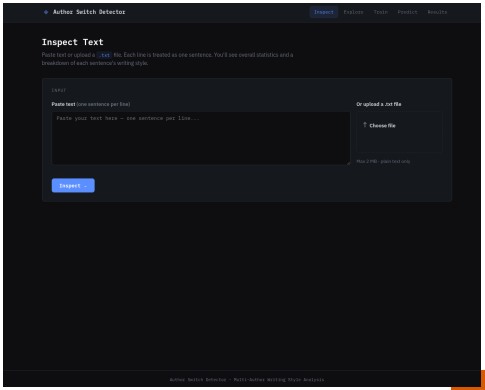
Functional Requirement Coverage

FR	Requirement	Prototype	Final
FR1	Upload or paste text	**Implemented**	**Implemented**
FR2	Sentence segmentation	**Implemented**	**Implemented**
FR3	Input validation and errors	**Implemented**	**Implemented**
FR4	Stylometric feature extraction	**Implemented**	**Implemented** (extra)
FR5	Pairwise feature differences	**Implemented**	**Implemented**
FR6	Author-switch prediction	**Implemented**	**Implemented**
FR7	Confidence and top features	**Partial**	**Implemented**
FR8	Admin model training	**Implemented**	**Implemented**
FR9	Model comparison	**Partial**	**Implemented**
Auth	User authentication	**Partial**	**Implemented**

Partial items are implemented with limited functionality in the prototype and will be fully realised in the final system.

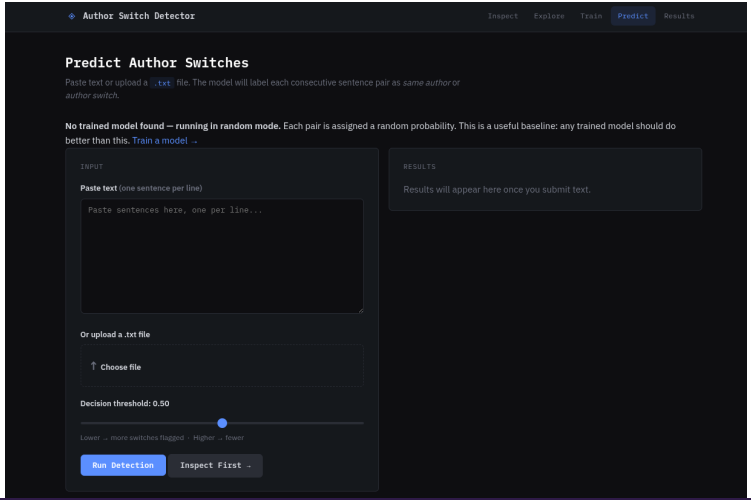
Prototype Screens – Home & Inspect





Left: Home page – model status and stylometric feature groups.
Right: Inspect page – per-sentence features and colour-coded style-distance bars (red = high stylistic distance).

Prototype Screens – Predict



Test Plan – Objectives & Levels

Main objectives

1. Validate all functional requirements against specifications.
2. Classifier macro-averaged F1 $>$ simple baseline on PAN 2026 held-out set.
3. Process 50-sentence documents within interactive time (target: < 10 s).
4. Display top stylometric features associated with predicted switches.
5. Graceful rejection of invalid/empty inputs.
6. Correct authentication validation.

Test Plan – Objectives & Levels

Main objectives

1. Validate all functional requirements against specifications.
2. Classifier macro-averaged F1 $>$ simple baseline on PAN 2026 held-out set.
3. Process 50-sentence documents within interactive time (target: < 10 s).
4. Display top stylometric features associated with predicted switches.
5. Graceful rejection of invalid/empty inputs.
6. Correct authentication validation.

Test levels (pyramid)

Unit	$\geq 80\%$ statement coverage; pytest with mocks.
Integration	Flask test client – upload, train, predict workflows.
System	End-to-end scenarios (multi-author, single-author, empty input).

Test Strategy

- **Approach:**
  - White-box: unit and integration tests (internal interfaces).
  - Black-box: system and acceptance tests (HTTP/UI interfaces).
- **Framework:** `pytest` + `unittest.mock`; Flask test client for fast, deterministic HTTP simulation.
- **Test Data:**
  - Unit tests: hand-crafted sentences with known feature values.
  - Integration / System: synthetic dataset with ground-truth labels.
  - Model evaluation: PAN 2026 held-out test split.

FR–Test Case Traceability

FR	Requirement	Covering Test Cases
FR1	Upload or paste text	UT-06–09, IT-01–02, ST-01
FR2	Sentence segmentation	UT-10–11
FR3	Input validation	UT-06, UT-08–09, UT-11, IT-03
FR4	Feature extraction	UT-12–16
FR5	Pairwise differences	UT-16
FR6	Author-switch prediction	UT-17–20, IT-04–06, ST-01–02
FR7	Confidence and explanation	ST-01, UAT
FR8	Model training	UT-21, IT-07–08, ST-04
FR9	Model comparison	UT-23, IT-11
Auth	Login authentication	UT-01–05

Preliminary Test Cases (Summary)

Category	IDs (Count)	Key Examples
Unit Tests (23)	UT-01 – UT-23	Authentication, document processing, feature extraction, prediction, training, evaluation
Integration (11)	IT-01 – IT-11	Inspect, Predict, Train, Explore, Compare endpoints via Flask test client
System (4)	ST-01 – ST-04	End-to-end: two switches, single-author, 50-sentence performance, train-then-predict
UAT	UAT	Proxy user performs upload-analyse-review; usability rating

Detailed

38-entry test case table provided in the PDR document.

Test Environment

Component	Details
Laptop Model	Acer Aspire A315-58
Operating System	Fedora Linux 44 (Workstation Edition)
Processor	11th Gen Intel Core i3-1115G4
Memory	8 GB RAM
Python Version	3.10+
Framework	Flask
ML Library	scikit-learn
Testing Framework	pytest
Browsers	Chrome 124+, Firefox 125+

Risks and Contingency

Risk	Likelihood	Mitigation
PAN dataset unavailable	Medium	Use synthetic dataset as drop-in replacement
Classifier performance below baseline	Medium	Ablation study; add features if needed
Time constraints limit UAT participants	High	One proxy instructor sufficient; structured walkthrough
Training too slow for CI pipeline	Low	Mock classifier in integration tests; nightly full training

Processing a Document in a Short Time

Performance Target

The system shall process documents containing up to **50 sentences** within a time frame suitable for interactive use.

Specific goal (from acceptance criteria):

- A fifty-sentence document must be processed in **under ten seconds** on reference hardware.

This requirement is validated by system test `ST-03` and ensures the tool remains practical for real-time interactive analysis.

Acceptance Criteria

The system is considered ready for the next development phase when:

1. All unit tests pass, with $\geq 80\%$ coverage in core processing modules.
2. All documented integration scenarios execute without runtime errors.
3. Preliminary classifier outperforms a simple baseline on the PAN 2026 held-out test split.
4. A fifty-sentence document is processed in < 10 **seconds** (performance requirement).
5. UAT proxy user completes the upload-analyse-review workflow without assistance and gives a usability rating $\geq 4/5$ on a Likert scale.

Prototype Implementation Details

- **Inspect page (FR1–FR4):** text upload, sentence segmentation, 31 stylometric features per sentence, overview statistics, colour-coded style-distance bar.
- **Explore page:** dataset statistics and example display.
- **Train page (FR8):** classifier selection (Logistic Regression, SVM, k-NN, MLP), cross-validation, synchronous training with live log streaming.
- **Predict page (FR5–FR7):** text submission, probability threshold slider, ML-based predictions when a model exists.
- **Results page:** held-out F1, precision, recall, cross-validation statistics.
- **Authentication (partial):** login form and session management; full validation deferred.

Known Limitations:

- Random-baseline predictor when no model is trained (UI exploration only).
- Synchronous training – will be moved to background task queue.

Conclusion & Future Work

Key Achievements

- Design review completed: modular architecture, conceptual class diagram, detailed test plan.
- Working proof-of-concept prototype validates the stylometric pipeline and user workflow.
- All core use cases (UC1–UC6) covered; FRs partially or fully implemented.

Conclusion & Future Work

Key Achievements

- Design review completed: modular architecture, conceptual class diagram, detailed test plan.
- Working proof-of-concept prototype validates the stylometric pipeline and user workflow.
- All core use cases (UC1–UC6) covered; FRs partially or fully implemented.

Future Work (Final Implementation)

- Asynchronous model training (background task queue).
- Support for PDF/DOCX uploads.
- Full user authentication and model comparison.
- Permutation-based feature importance ranking.

References I

[1] G. J. Myers, T. Badgett, T. M. Thomas, and C. Sandler.
The Art of Software Testing, 2nd ed.
John Wiley & Sons, 2004.

[2] E. Zangerle, M. Potthast, and B. Stein.
PAN 2026 shared task on multi-author writing style change detection.
In *Working Notes of CLEF*, 2026.

Thank You

Questions?

223004635@student.uj.ac.za